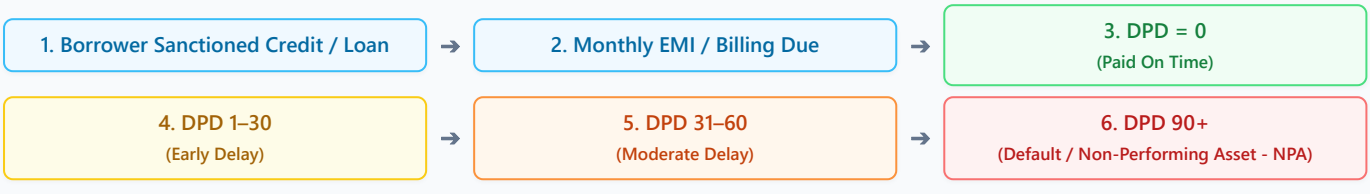
Credit Delinquency Early Warning System (EWS)

Student Quick Revision Guide — Visual Flow Diagrams & Code-Verified Technical Details

📌 1. Problem Statement, Existing Ideas & Issues, and Project Idea

🔹 Step 1: The Problem Statement (Why Banks Need EWS)

When borrowers take credit cards or loans, they pay monthly EMIs. If a customer faces financial distress, payments become delayed. In banking, delays are measured using **Days Past Due (DPD)**.



- **Days Past Due (DPD):** Number of days elapsed past payment due date.
- **Non-Performing Asset (NPA):** Loan classified as default when unpaid for 90+ days. The bank suffers financial loss.
- **The Banking Problem:** Catching default at DPD 90 is too late! Money recovery post-default is expensive and yields low recovery rates.

🔹 Step 2: Existing Ideas & Technical Issues (Why Naive Models Fail)

Standard credit scoring models and naive ML scripts suffer from 6 critical technical flaws:

1. Data Leakage
Training on post-event fields like `current_dpd`, `recovery_lag_days`, or raw `behavioral_risk_score` gives fake 100% test accuracy that crashes in live production.

2. Division-by-Zero Crashes
Naive financial ratios like `emi_amount / net_disposable_income` crash with zero/missing denominators without safety protection.

3. Unhandled Class Imbalance
Delinquency rate is ~10-15%. Unweighted models maximize raw accuracy by predicting 'No Default' for every customer.

4. Uncalibrated Output Scores
Tree-based output scores push to extremes and do not represent real-world empirical probability percentages.

5. Arbitrary 0.5 Decision Cutoff
Using default 0.5 threshold under class imbalance misses up to 80% of true delinquent accounts (high False Negative rate).

6. Black-Box Predictions
Models output risk flags without providing specific customer-level reason codes required for credit officers to act.

🔹 Step 3: The Project Idea & Solution Strategy

Our **Early Warning System (EWS)** predicts delinquency risk **30 to 90 days in advance** using safe behavioral signals, calibrated ML, and business rules.

- **Leakage-Safe Features:** Programmatically drops leaky features (`current_dpd`, `recovery_lag_days`, `behavioral_risk_score`) using `drop_leaky()`.
- **Safe Ratio Engineering:** Uses `eps = 1e-9` safety factor in all ratio denominators to prevent zero-division crashes (`build_safe_features`).
- **Champion Classifier:** Trains LightGBM with `scale_pos_weight = (y==0).sum() / (y==1).sum()` to pay attention to minority default cases.
- **Probability Calibration:** Fits Isotonic Regression (`CalibratedClassifierCV`) on held-out split (`X_test1`) so output scores equal true empirical probabilities.
- **Constrained Threshold Tuning:** Sweeps thresholds from 0.05 to 0.40 on test set (`File02`) to maximize F1-score with Recall ≥ 0.70.
- **Business Rule Overlay:** Combines model probabilities with business risk rules (`ews_overlay`).
- **SHAP Explainability:** Computes `shap.TreeExplainer` to extract top-5 customer-level reason codes (`↑ risk` / `↓ risk`).

📌 2. Tech Stack & What Each Tool is Doing in the Project

Tool / Library	Role in Project	What It Is Doing in the Codebase
Python 3.9+	Core Programming Language	Executes the end-to-end data processing, feature engineering, modeling, calibration, threshold tuning, and explainability script.
Pandas	Data Processing & Feature Storage	Loads raw CSV datasets (`File01` , `File02` , `File03`), manages DataFrames, drops leaky columns, creates ratio features, and converts categorical columns (`employment_type` , `marital_status` , `loan_type` , `income_variability_flag` , `partial_payment_flag` , `utilization_shock_flag`) to `category` dtype.
NumPy	Numerical Math & Safety Factor	Applies logarithmic scaling (`np.log1p` for `log_income`), adds `eps = 1e-9` safety term to prevent zero-division, computes 75th percentiles (`np.nanpercentile`), and sorts array indices (`np.argsort`) for top-5 SHAP reason codes.
LightGBM	Champion ML Classifier	Trains `lgb.LGBMClassifier(n_estimators=1200, learning_rate=0.03, num_leaves=31, scale_pos_weight=pos_weight)` to handle imbalanced credit data fast with native categorical handling.
Scikit-Learn	Preprocessing, Calibration & Metrics	Performs stratified 70/30 split (`train_test_split`), probability calibration (`CalibratedClassifierCV` with Isotonic regression on `X_test1_2`), trains Logistic Regression challenger (SAGA solver), and evaluates metrics (ROC-AUC, KS, Brier Score Loss).
SHAP	Model Explainability	Uses `shap.TreeExplainer(base_clf)` to compute feature impact values and extract top-5 customer-level reason codes.
Matplotlib / Seaborn	Visualization	Renders feature distribution plots during exploratory data analysis and plots global SHAP summary visualizations.

📌 3. Project Architecture Diagram & Internal Working

🔹 Component & Workflow Architecture Diagram

1. Data Ingestion Component — Input Files
- `File01_Delinquency_ews_Model.csv` (80,000 rows - Training & Calibration)
- `File02_Delinquency_ews_20k_1_Test_Model.csv` (20,000 rows - Threshold Tuning)
- `File03_Delinquency_ews_20k_2_Bus_Validate.csv` (20,000 rows - Validation Inference)

⬇

2. Preprocessing & Safe Feature Engineering Component — Data Cleaning & Safe Ratios
- Perform 70/30 Stratified Split on File01 ➔ `X_train` (56k rows) and `X_test1` (24k rows)
- Drop Leaky Columns via `drop_leaky()` : `current_dpd` , `recovery_lag_days` , `behavioral_risk_score`
- Calculate Safe Ratios via `build_safe_features()` using `eps = 1e-9` safety factor
- Convert `cat_cols` to Pandas `category` dtype for native LightGBM splitting

⬇

3. Model Training & Probability Calibration Component — ML Algorithms & Calibration
- Train Base Champion `LGBMClassifier(n_estimators=1200, scale_pos_weight=pos_weight)`
- Calibrate Probabilities via `CalibratedClassifierCV(FrozenEstimator(base_clf), method="isotonic", cv=2)` on `X_test1_2`
- Train Logistic Regression Challenger Pipeline (StandardScaler + OneHotEncoder + SAGA solver)
- Train No-Lag Validation Model via `drop_lag_features()` to verify leakage safety

⬇

4. Decision Engine Component — Threshold Sweep & Overlay
- Run `threshold_sweep()` on File02 (X_test2_2, y_test2_2) to find threshold maximizing F1 s.t. Recall ≥ 0.70
- Apply `ews_overlay()` rule engine: Flag if Probability ≥ optimal threshold **OR** if moderate probability (≥ 0.12) coincides with high utilization (≥ 75%), high payment volatility (≥ 75th percentile), and bounce (≥ 1) or EMI stress

⬇

5. Explainability & Output Component — SHAP & Final CSV
- Calculate local SHAP values using `shap.TreeExplainer(base_clf)`
- Extract Top-5 Customer Reason Codes sorted by `np.argsort(np.abs(vals))[::-1][:5]` (`↑ risk` if SHAP > 0, `↓ risk` if SHAP < 0)
- Export final submission dataframe containing `customer_id` and binary `ews_flag` to `File04_Delinquency_Results_Submit_Final.csv`

🔹 Internal Working: Step-by-Step Workflow (Visual Revision Cards)

📥 Stage 1: Data Ingestion & Stratified Splitting `Code: train_test_split()`

• Loads `File01_Delinquency_ews_Model.csv` (80,000 rows, 40 columns).
• Performs a 70/30 stratified split: `X_train` / `y_train` (56,000 rows for model training) and `X_test1` / `y_test1` (24,000 rows held out specifically for **Probability Calibration**).

🛡️ Stage 2: Preprocessing & Safe Feature Engineering `Code: drop_leaky() & build_safe_features()`

• **Drops Leaky Columns:** `current_dpd`, `recovery_lag_days`, `behavioral_risk_score` to prevent fake 100% test scores.
• **Log Income Transform:** `log_income = np.log1p(monthly_gross_income)`.
• **10+ Safe Ratios with `eps = 1e-9` safety term:** `emi_stress_ratio_safe`, `bounce_frequency_ratio_safe`, `outstanding_balance_ratio_safe`, `utilization_shock_safe`, `rolling_dpd_trend_safe`, `delinquency_acceleration_safe`, `payment_volatility_safe`, `pay_to_due_ratio_3m_safe`.
• **Categorical Conversion:** Converts `cat_cols` to Pandas `category` dtype.

⚡ Stage 3: Model Training & Probability Calibration `Code: LGBMClassifier() & CalibratedClassifierCV()`

• **Champion Model:** Fits `base_clf = lgb.LGBMClassifier(n_estimators=1200, learning_rate=0.03, num_leaves=31, scale_pos_weight=pos_weight)` where `pos_weight = (y_train2 == 0).sum() / (y_train2 == 1).sum()`.
• **Probability Calibration:** Fits `cal_clf = CalibratedClassifierCV(FrozenEstimator(base_clf), method="isotonic", cv=2)` on `X_test1_2` so predicted scores equal true background default probabilities.
• **Challengers:** Fits Logistic Regression pipeline (StandardScaler + OneHotEncoder + SAGA) and No-Lag model (using `drop_lag_features`).

🎯 Stage 4: Threshold Sweeping & Business Overlay `Code: threshold_sweep() & ews_overlay()`

• **Threshold Sweeping:** Sweeps thresholds `[0.05 ... 0.40]` on `File02` (20,000 rows) to select optimal cutoff maximizing F1-score with Recall ≥ 0.70.
• **Business Overlay Rule (`ews_overlay`):** Flags account if:

```
Model Probability ≥ optimal_threshold OR
(Probability ≥ 0.12 AND utilization ≥ 75% AND volatility ≥ 75th_pct AND (bounce ≥ 1 OR emi_stress ≥ 75th_pct))
```

💡 Stage 5: SHAP Reason Codes & Output CSV Export `Code: shap.TreeExplainer() & to_csv()`

• **SHAP Values:** Computes `explainer = shap.TreeExplainer(base_clf)` on individual customer rows.
• **Reason Codes:** Ranks top-5 features by `np.argsort(np.abs(vals))[::-1][:5]` and assigns risk direction (`↑ risk` if SHAP > 0, `↓ risk` if SHAP < 0).
• **Export Output:** Writes customer dataframe to `File04_Delinquency_Results_Submit_Final.csv` with `customer_id` and binary `ews_flag`.